

Lab 1: Python Essentials and basic data processing

1 Python development kits

For python beginners, access to <https://github.com/Akuli/python-tutorial> for a python programming tutorial.

1.1 Anaconda

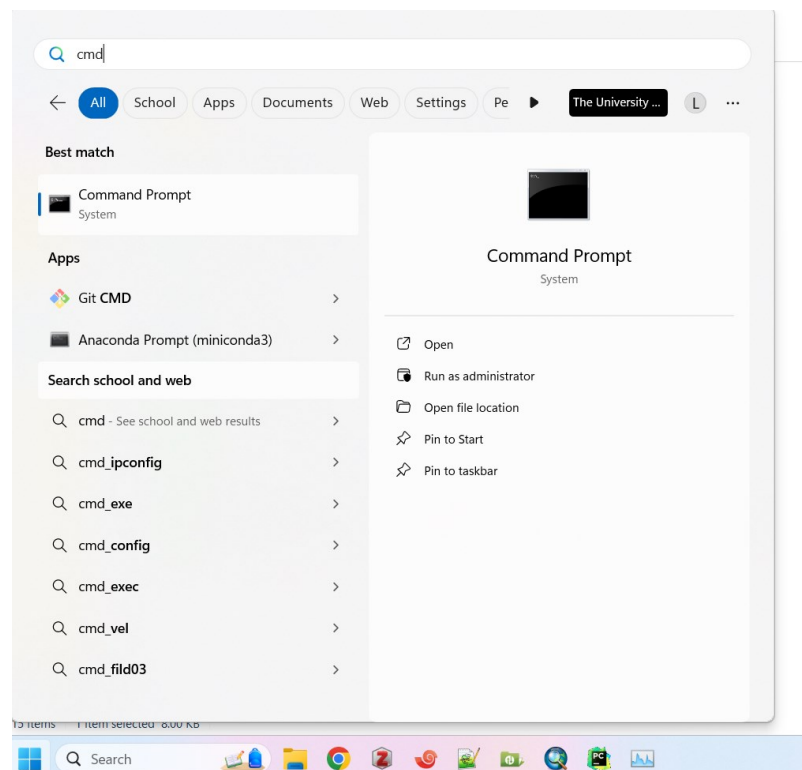
Anaconda provides an ensemble python development environment for scientific calculation, machine learning and deep learning in an easy manner. It includes common used python libraries, Numpy, Matplotlib and PIL.

If you are familiar with Anaconda and python, you can just skip this part.

If not, please follow the bellow instructions to get some information about Anaconda.

1.1.1 Check whether it has successfully installed

There are many ways to open the Command Prompt. A simple way is to open your start menu on your computer and type “cmd”. It may be like as follows:



Open the Command Prompt then. Type command “conda -V”. The outputs is the version number of the anaconda.

```
Command Prompt
Microsoft Windows [Version 10.0.22631.3737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\wenyuan>conda -V
conda 23.10.0

C:\Users\wenyuan>
```

It demonstrates that Anaconda has been successfully installed in your computer.

1.1.2 Get the info of installed python libraries

The anaconda contains many python libraries which are introduced in the first part of this course. Now you can type “conda list” to get these libraries’ information.

```
Command Prompt

pip                23.3                py311haa95532_0
pluggy            1.0.0                py311haa95532_1
powershell_shortcut_miniconda 0.0.1                haa95532_1
pybind11-abi       4                    hd3eb1b0_1
pycosat            0.6.6                py311h2bbff1b_0
pyparser           2.21                 pyhd3eb1b0_0
pyopenssl          23.2.0               py311haa95532_0
pysocks            1.7.1                py311haa95532_0
python             3.11.5               he1021f5_0
reproc             14.2.4               hd77b12b_1
reproc-cpp         14.2.4               hd77b12b_1
requests           2.31.0               py311haa95532_0
ruamel.yaml        0.17.21              py311h2bbff1b_0
setuptools         68.0.0               py311haa95532_0
sqlite             3.41.2               h2bbff1b_0
tk                 8.6.12               h2bbff1b_0
tqdm               4.65.0               py311h746a85d_0
truststore         0.8.0                py311haa95532_0
tzdata             2023c                h04d1e81_0
urllib3            1.26.18              py311haa95532_0
vc                 14.2                 h21ff451_1
vs2015_runtime     14.27.29016          h5e58377_2
wheel              0.41.2               py311haa95532_0
win_inet_pton      1.1.0                py311haa95532_0
xz                 5.4.2                h8cc25b3_0
yaml-cpp           0.8.0                hd77b12b_0
zlib               1.2.13               h8cc25b3_0
zstandard          0.19.0               py311h2bbff1b_0
zstd               1.5.5                hd43e919_0

C:\Users\wenyuan>
```

1.1.3 Install new library

We often need to install some new libraries.

Type “conda install xx” to install the library you need.

Now, we need to install “tiff file” to process geospatial data with tiff format.

Type “conda install tiff file”

```
Command Prompt - conda in  ×  +  ▾

libbrotlicommon    pkgs/main/win-64::libbrotlicommon-1.0.9-h2bbff1b_8
libbrotlidec       pkgs/main/win-64::libbrotlidec-1.0.9-h2bbff1b_8
libbrotlienc       pkgs/main/win-64::libbrotlienc-1.0.9-h2bbff1b_8
libdeflate         pkgs/main/win-64::libdeflate-1.17-h2bbff1b_1
libpng             pkgs/main/win-64::libpng-1.6.39-h8cc25b3_0
libtiff            pkgs/main/win-64::libtiff-4.5.1-hd77b12b_0
libwebp-base       pkgs/main/win-64::libwebp-base-1.3.2-h2bbff1b_0
libzopfli          pkgs/main/win-64::libzopfli-1.0.3-ha925a31_0
mkl                pkgs/main/win-64::mkl-2023.1.0-h6b88ed4_46358
mkl-service        pkgs/main/win-64::mkl-service-2.4.0-py311h2bbff1b_1
mkl_fft           pkgs/main/win-64::mkl_fft-1.3.10-py311h827c3e9_0
mkl_random         pkgs/main/win-64::mkl_random-1.2.7-py311hea22821_0
numpy             pkgs/main/win-64::numpy-1.26.4-py311hdab7c0b_0
numpy-base        pkgs/main/win-64::numpy-base-1.26.4-py311hd01c5d8_0
openjpeg          pkgs/main/win-64::openjpeg-2.5.2-hae555c5_0
platformdirs      pkgs/main/win-64::platformdirs-3.10.0-py311haa95532_0
snappy            pkgs/main/win-64::snappy-1.2.1-hcdb6601_0
tbb               pkgs/main/win-64::tbb-2021.8.0-h59b6b97_0
tiffiff           pkgs/main/win-64::tiffiff-2023.4.12-py311haa95532_0
zfp               pkgs/main/win-64::zfp-1.0.0-hd77b12b_0

The following packages will be UPDATED:

ca-certificates    2023.08.22-haa95532_0 --> 2024.7.2-haa95532_0
certifi            2023.7.22-py311haa95532_0 --> 2024.8.30-py311haa95532_0
conda              23.10.0-py311haa95532_0 --> 23.11.0-py311haa95532_0
openssl            3.0.12-h2bbff1b_0 --> 3.0.15-h827c3e9_0
vs2015_runtime     14.27.29016-h5e58377_2 --> 14.40.33807-h98bb1dd_0

Proceed ([y]/n)? |
```

Press Enter to complete the installation.

1.1.4 Python

Type “python” in the command prompt and a python interpreter is opened.

```
Command Prompt - python  ×  +  ▾  -  □  ×

Microsoft Windows [Version 10.0.22631.3737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\wenyuan>python
Python 3.11.5 | packaged by Anaconda, Inc. | (main, Sep 11 2023, 13:26:23) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

You can type some python scripts line by line as follows to get current time:

```
import datetime
datetime.datetime.now()
```

```
Command Prompt - python
Microsoft Windows [Version 10.0.22631.3737]
(c) Microsoft Corporation. All rights reserved.

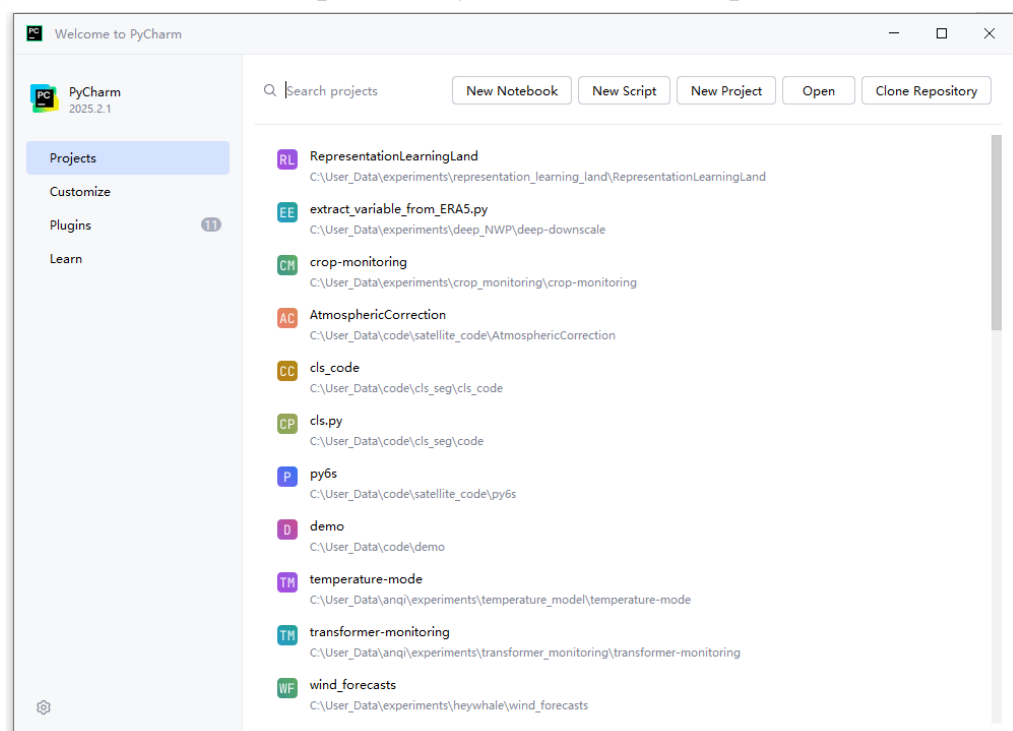
C:\Users\wenyuan>python
Python 3.11.5 | packaged by Anaconda, Inc. | (main, Sep 11 2023, 13:26:23) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import datetime
>>> datetime.datetime.now()
datetime.datetime(2024, 9, 4, 16, 41, 3, 228246)
>>> |
```

1.2 PyCharm

PyCharm is an integrated development environment (IDE) which can help us to manage our python codes.

Now follow the instructions to build new python project and run python. Our remaining labs are all implemented with PyCharm.

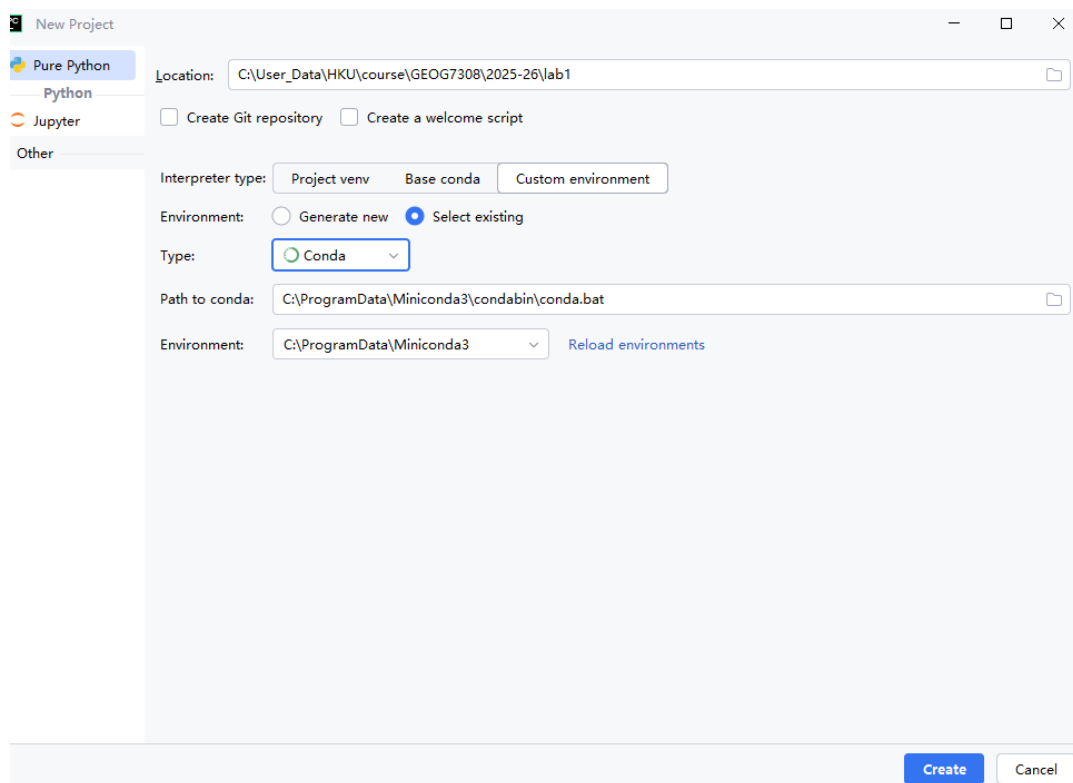
1) Double-click and open the PyCharm on desktop



2) Click “New Project” to create a new python project.

A python project is a collection of python scripts. Each python script is a file with suffix “py”.

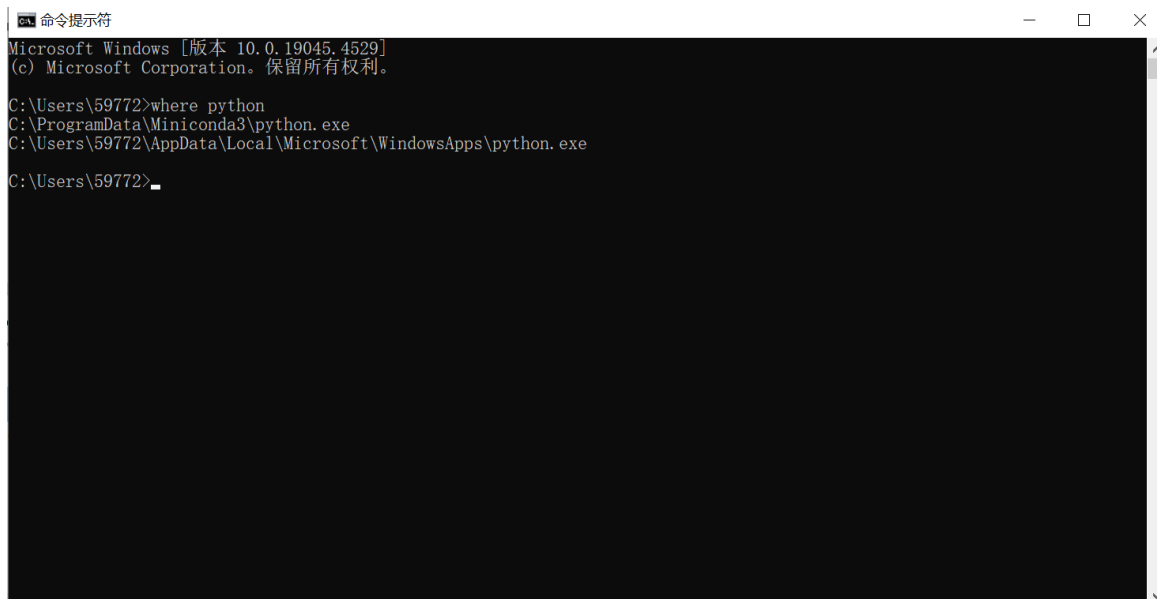
The configurations in the “new project” dialogue are as follows:



location: where to save your project and codes.

Interpreter type: set it as “custom environment” -> “select existing”

path to conda: the location of the python. It is usually set automatically. If not, just open the command prompt, and type “where python”. It will tell you the locations of python in this computer, select the python which is under the conda-like folder (eg, miniconda or anaconda). After get the location of python, you can set it into this item manually.



```
命令提示符
Microsoft Windows [版本 10.0.19045.4529]
(c) Microsoft Corporation。保留所有权利。

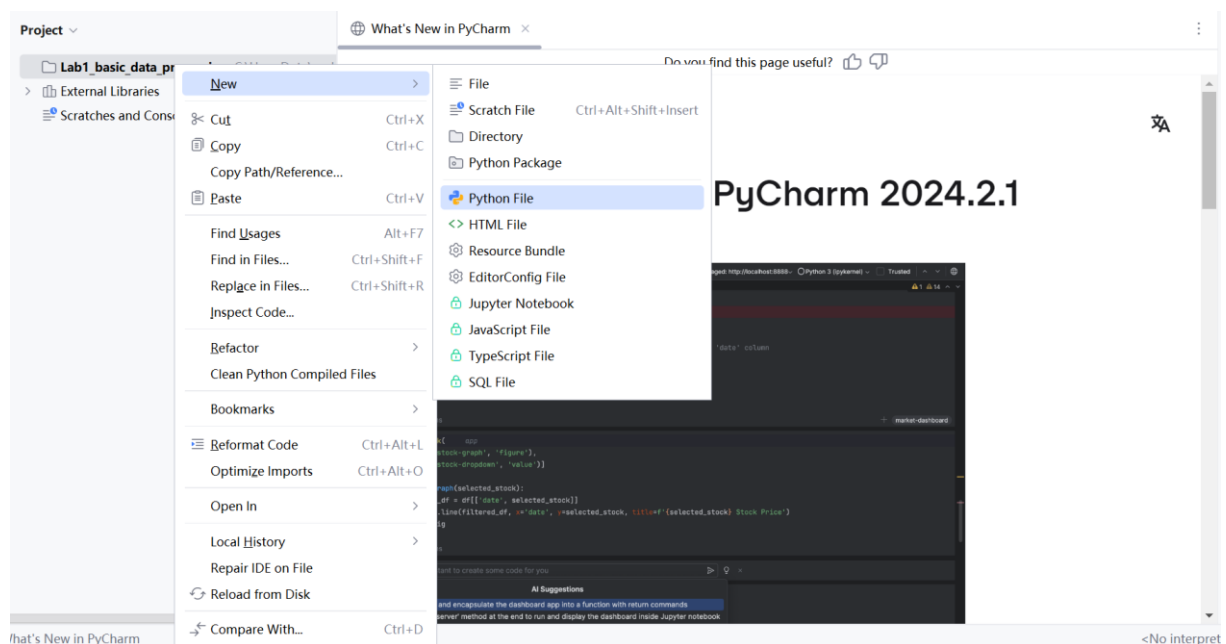
C:\Users\59772>where python
C:\ProgramData\Miniconda3\python.exe
C:\Users\59772\AppData\Local\Microsoft\WindowsApps\python.exe

C:\Users\59772>
```

Make sure all items are set correctly, then click “Create” and you will get a PyCharm project for python development.

3) New python file

Right-click on the project name. In the context menu, select “new” -> “Python file”.

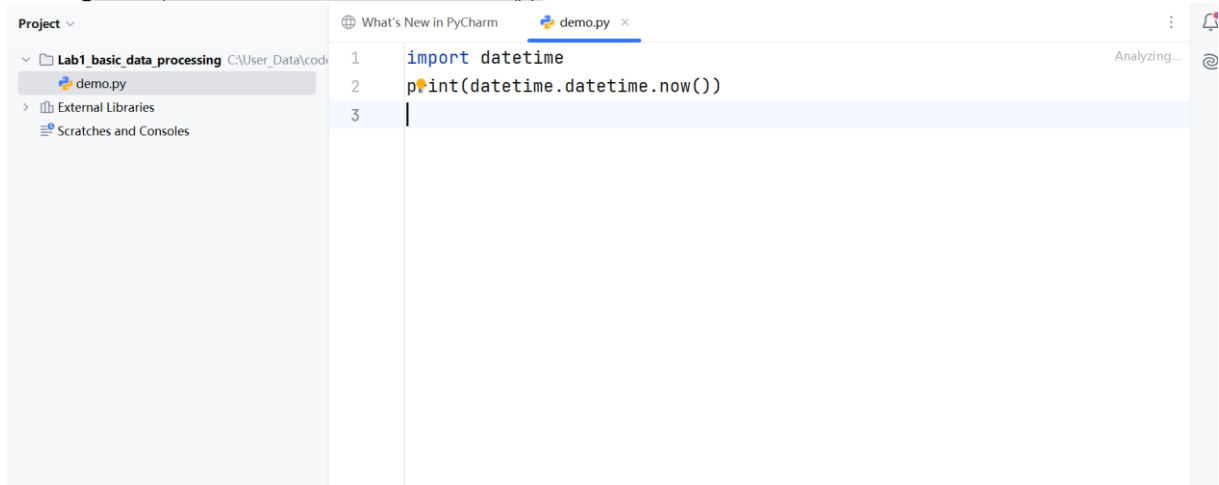


Name the new python file as “demo”, then we get a new python file named “demo.py”.

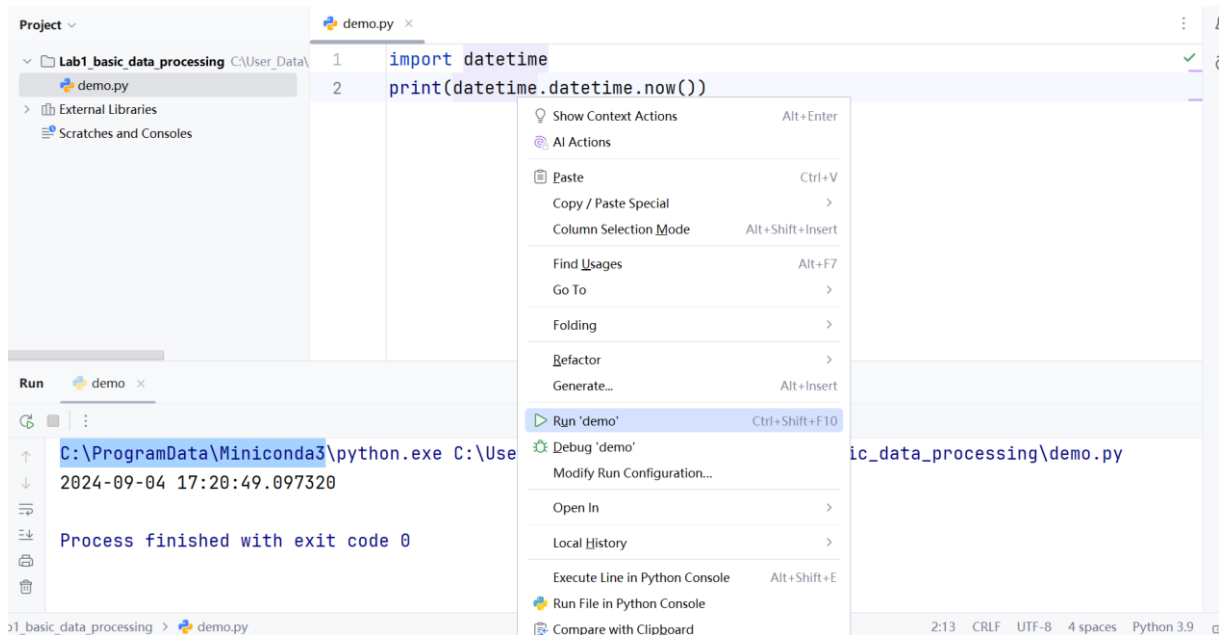
4) Run python

Type the python scripts that gets the current time into the demo.py. Different with that in command prompt, we need to use python command “print” to display the output.

```
import datetime
print(datetime.datetime.now())
```



Right-click on the content part of demo.py. In the context menu, select “run demo.py” to get the outputs.



2 Python Essentials

This section introduces foundational Python skills tailored to geospatial data workflows (e.g., handling raster/vector data, automating spatial analysis). All exercises are completed in PyCharm—create a file named `geospatial_python_fundamentals.py` to practice.

2.1 Basic Data Types

Python's basic data types serve as the foundational building blocks for organizing and manipulating data—especially critical for geospatial workflows, where structured storage of spatial attributes (e.g., image dimensions, GPS coordinates, pixel values, or metadata) directly impacts processing efficiency and accuracy. These core types are designed to handle distinct data needs: integers (int) for counting discrete values like the number of bands in a satellite image or image pixel dimensions; floats (float) for precise continuous data such as latitude/longitude coordinates, elevation, or pixel reflectance; strings (str) for text-based information like file paths to geospatial datasets or band names; lists (list) for ordered, modifiable collections (e.g., a sequence of monthly image paths or band labels); tuples (tuple) for fixed, unchangeable values like bounding box coordinates or image resolution (to prevent accidental modification); and dictionaries (dict) for key-value pairs that organize hierarchical metadata (e.g., linking a band name to its wavelength or an image's acquisition date to its sensor type).

Data Type	Primary Use in Geospatial Work	Key Characteristics	Example
Integer (int)	Counting (e.g., number of bands, image rows/columns), GPS coordinate decimals (truncated)	Whole numbers, no decimal points	<code>band_count = 12,</code> <code>img_width = 1024</code>
Float (float)	Continuous values (e.g., pixel reflectance, latitude/longitude, elevation)	Includes decimal points for precision	<code>reflectance = 0.68,</code> <code>latitude</code> <code>= 40.7128</code>
String (str)	File paths, data labels (e.g., band names, dataset IDs), attribute names	Text-based, enclosed in quotes	<code>data_path =</code> <code>"C:/geodata/sat_img.tif",</code> <code>band_name = "NIR"</code>
List (list)	Ordered, mutable collections (e.g., band names, pixel values for a row, list of image paths)	Modifiable (add/remove items), uses <code>[]</code>	<code>band_names = ["Red",</code> <code>"Green", "Blue"],</code> <code>row_pixels = [210, 205,</code> <code>198]</code>
Tuple (tuple)	Fixed, immutable values (e.g., image dimensions, GPS coordinate pairs,	Unmodifiable (prevents accidental changes), uses <code>()</code>	<code>img_dimensions = (1024,</code> <code>768),</code> <code>gps_point =</code> <code>(40.7128, -74.0060)</code>

	bounding boxes)		
Dictionary (dict)	Key-value pairs for metadata (e.g., image resolution, acquisition date, band wavelengths)	Keys are unique; values can be any type, uses {}	<pre>img_metadata = {"resolution": 30, "acquisition_date": "2024- 01-15", "band_wavelengths": {"Red": 650, "Blue": 490}}</pre>

Here is a practice for Implementing Data Types for Geospatial Scenarios.

Write code in PyCharm to define and manipulate the data types above, aligned with real-world geospatial tasks.

```
# 1. Integer: Count bands and image dimensions
band_count = 8 # 8-band multispectral image
img_height = 768 # Image height in pixels
img_width = 1024 # Image width in pixels

# 2. Float: Store spatial coordinates and pixel reflectance
site_lat = 37.7749 # Latitude of San Francisco (example)
site_lon = -122.4194 # Longitude of San Francisco (example)
nir_reflectance = 0.723 # Reflectance of NIR band (0–1 range)

# 3. String: Define file paths and data labels
vector_file_path = "C:/geodata/shapefiles/urban_areas.shp"
raster_file_path = "C:/geodata/rasters/land_cover.tif"
dataset_id = "Landsat-9_OLI_TIRS_2024"

# 4. List: Store band names and multiple image paths
sentinel_bands = ["B1 (Coastal)", "B2 (Blue)", "B3 (Green)", "B4 (Red)", "B8 (NIR)"]
image_paths = [
"C:/geodata/20240105_img.tif",
"C:/geodata/20240205_img.tif",
"C:/geodata/20240305_img.tif"
]

# 5. Tuple: Define fixed bounding box (min_lon, min_lat, max_lon, max_lat)
study_area_bbox = (-122.519, 37.701, -122.319, 37.819) # San Francisco bounding box
pixel_size = (30, 30) # 30m x 30m pixel size (width, height)
```

```
# 6. Dictionary: Store image metadata (key-value pairs)
image_metadata = {
    "sensor": "Landsat-9 OLI/TIRS",
    "acquisition_date": "2024-03-10",
    "resolution_m": 30,
    "band_info": {
        "Red": {"wavelength_nm": 650, "data_type": "float32"},
        "NIR": {"wavelength_nm": 860, "data_type": "float32"}
    }
}

# Print selected values to verify (focus on geospatial relevance)
print("Study Area Bounding Box:", study_area_bbox)
print("NIR Band Wavelength:", image_metadata["band_info"]["NIR"]["wavelength_nm"], "nm")
print("Monthly Image Paths (2024):", image_paths)
```

2.2 Control Flow: Loops & Conditional Statements

Control flow is what makes Python code go beyond simple “one-step-after-another” execution—it lets your program make choices and repeat tasks automatically, which is essential for writing efficient, adaptable code in any scenario. Conditional statements (like if-elif-else) help your code “decide” what to do next: for example, checking if a number is positive or negative, or if a file exists before opening it. Loops (like for or while) handle repetitive work without you rewriting code—such as calculating values for every item in a list, or running a task 10 times in a row. Together, these tools turn basic code into something that can adapt to different data and solve real-world problems.

Practice 1: Loops for Batch Geospatial Tasks

Use loops to process multiple image paths and extract band information:

```
# Example 1: Iterate over image paths to print file names and acquisition months
print("\n--- Batch Image Path Processing ---")
for path in image_paths:
    # Extract file name from path (split by "/" and take last element)
    file_name = path.split("/")[-1]
    # Extract month from file name (assumes format: "2024MMDD_img.tif")
```

```

    month = file_name[4:6] # Characters 5–6 (0-indexed) are month
    print(f"Image File: {file_name} | Acquisition Month (2024): {month}")
# Example 2: Iterate over band names with indices (for band-specific analysis)
print("\n--- Band Index and Name Mapping ---")
band_number=0
while band_number<len(sentinel_bands):
    band=sentinel_bands[band_number]
    print(f"Band Number {band_number}: {band}")
    band_number=band_number+1

```

Practice 2: Conditional Statements for Geospatial Filtering

Use if-elif-else to classify pixels, filter images, or validate metadata:

```

# Example 1: Classify pixel quality (cloud/shadow/clear) for geospatial imagery
def classify_pixel_quality(pixel_value):
    """Classify a pixel based on brightness (common in cloud detection)."""
    if pixel_value > 180: # Bright pixels = likely cloud
        return "Cloud"
    elif pixel_value < 50: # Dark pixels = likely shadow
        return "Shadow"
    else: # Moderate brightness = clear land/water
        return "Clear"
# Test with sample pixel values (simulated from a visible band)
sample_pixels = [205, 42, 130, 190, 65]
print("\n--- Pixel Quality Classification ---")
for pixel in sample_pixels:
    quality = classify_pixel_quality(pixel)
    print(f"Pixel Value: {pixel} | Classification: {quality}")
# Example 2: Filter images by acquisition month (e.g., select summer months)
summer_months = ["06", "07", "08"] # June, July, August (2-digit format)
summer_image_paths = []
print("\n--- Filtering Summer Images ---")
for path in image_paths:
    file_name = path.split("/")[-1]
    month = file_name[4:6]
    if month in summer_months:
        summer_image_paths.append(path)
        print(f"Summer Image: {file_name}")
# Example 3: Validate image resolution (ensure it matches study requirements)
required_resolution = 30 # Study requires 30m resolution
actual_resolution = image_metadata["resolution_m"]
print("\n--- Resolution Validation ---")

```

```
if actual_resolution == required_resolution:
    print(f"Image resolution ({actual_resolution}m) matches required resolution ({required_resolution}m).")
elif actual_resolution < required_resolution:
    print(f"Image resolution ({actual_resolution}m) is higher than required ({required_resolution}m) — downsampling may be needed.")
else:
    print(f"Image resolution ({actual_resolution}m) is lower than required ({required_resolution}m) — data is insufficient.")
```

2.3 Functions

Functions encapsulate repetitive tasks (e.g., calculating image statistics, converting coordinates) into reusable blocks. This reduces code duplication and ensures consistency.

Write functions to compute band statistics, validate file paths, and convert coordinate formats:

```
# Example 1: Calculate basic statistics for a geospatial band (mean, min, max)
def calculate_band_stats(pixel_values):
    """
    Compute mean, minimum, and maximum values for a geospatial band.

    Args:
        pixel_values (list): List of pixel values from a single band.

    Returns:
        dict: Dictionary containing mean, min, and max values.
    """
    if not pixel_values: # Handle empty list (avoid errors)
        return {"mean": 0.0, "min": 0.0, "max": 0.0}

    band_mean = sum(pixel_values) / len(pixel_values)
```

```

band_min = min(pixel_values)
band_max = max(pixel_values)

# Round to 2 decimal places (standard for geospatial reporting)
return {
    "mean": round(band_mean, 2),
    "min": round(band_min, 2),
    "max": round(band_max, 2)
}

# Test with simulated Red band values (agricultural area)
red_band_pixels = [110, 125, 108, 132, 115, 120, 118, 128, 112, 123]
red_band_stats = calculate_band_stats(red_band_pixels)
print("\n--- Red Band Statistics (Agricultural Area) ---")
print(f"Mean: {red_band_stats['mean']}, Min: {red_band_stats['min']}, Max: {red_band_stats['max']}")

# Example 2: Validate if a geospatial file path ends with a valid extension
def is_valid_geospatial_path(file_path, valid_extensions):
    """
    Check if a file path has a valid geospatial extension (e.g., .tif, .shp).

    Args:
        file_path (str): Path to the file to validate.
        valid_extensions (list): List of valid file extensions (e.g., [".tif", ".shp"]).

    Returns:
        bool: True if valid, False otherwise.
    """

    # Get file extension (split by "." and take last element)
    file_ext = file_path.lower().split(".")[1]

    # Check if extension (with ".") is in valid_extensions
    return f".{file_ext}" in valid_extensions

# Define valid extensions for geospatial data
valid_geo_extensions = [".tif", ".tiff", ".shp", ".geojson", ".gpkg"]

```

```

# Test with sample paths
test_paths = [vector_file_path, raster_file_path, "C:/data/report.pdf"]
print("\n--- Geospatial File Path Validation ---")
for path in test_paths:
    is_valid = is_valid_geospatial_path(path, valid_geo_extensions)
    status = "VALID" if is_valid else "INVALID"
    print(f"Path: {path} | Status: {status}")

# Example 3: Convert decimal degrees to degrees-minutes-seconds (DMS) for GPS
def decimal_degrees_to_dms(decimal_value, coordinate_type):
    """
    Convert decimal degrees (DD) to degrees-minutes-seconds (DMS) format.

    Args:
        decimal_value (float): Latitude or longitude in decimal degrees.
        coordinate_type (str): "lat" for latitude, "lon" for longitude.

    Returns:
        str: DMS string (e.g., "40° 42' 53.688" N").
    """
    # Handle negative values (south for lat, west for lon)
    sign = ""
    if decimal_value < 0:
        decimal_value = abs(decimal_value)
        sign = "S" if coordinate_type == "lat" else "W"
    else:
        sign = "N" if coordinate_type == "lat" else "E"

    # Calculate degrees, minutes, seconds
    degrees = int(decimal_value)
    minutes = int((decimal_value - degrees) * 60)
    seconds = (decimal_value - degrees - minutes/60) * 3600

```

```

    return f"{degrees}° {minutes}' {seconds:.3f}" {sign}"
# Convert sample coordinates (San Francisco)
lat_dms = decimal_degrees_to_dms(site_lat, "lat")
lon_dms = decimal_degrees_to_dms(site_lon, "lon")
print("\n--- Coordinate Conversion (Decimal Degrees → DMS) ---")
print(f"Latitude (DD): {site_lat} → DMS: {lat_dms}")
print(f"Longitude (DD): {site_lon} → DMS: {lon_dms}")

```

2.4 Python Packages

Python’s core functionality is limited—packages (libraries) extend it to handle specialized geospatial tasks (e.g., reading raster data, performing spatial analysis, visualizing maps). Mastering key packages is critical for efficient geospatial workflows.

Key Packages for Geospatial Work:

Package	Primary Use	Example Tasks
<code>numpy</code>	Numerical computing (core for all geospatial data)	Create/process raster arrays, calculate statistics
<code>pandas</code>	Tabular data handling	Manage attribute tables (e.g., vector feature attributes)
<code>geopandas</code>	Vector data processing	Read/write shapefiles, perform spatial joins, buffer analysis
<code>rasterio</code>	Raster data processing	Read/write TIFF files, handle geospatial metadata (CRS, transform)
<code>matplotlib</code>	Data visualization	Plot raster images, vector features, and statistical charts
<code>scikit-learn</code>	Machine learning	Classify land cover, predict spatial patterns

2.4.1 Practice: Import and Use Numpy

NumPy (Numerical Python) is the backbone of geospatial data processing. It provides efficient, multi-dimensional arrays (ndarray) to store raster data (e.g., satellite images) and tools to manipulate these arrays (e.g., slicing regions of interest, calculating statistics). Without NumPy, handling large geospatial datasets would be slow and inefficient.

```
# Import required packages (ensure they are installed via conda/pip)
import numpy as np # Numerical computing (raster arrays)

# Example 1: Create a 2D array (single-band raster: 5 rows x 5 columns)
# Simulate a digital elevation model (DEM) with elevation values (0–500 meters)
dem_array = np.array([
    [120, 135, 150, 145, 130],
    [130, 145, 160, 155, 140],
    [140, 155, 170, 165, 150],
    [135, 150, 165, 160, 145],
    [125, 140, 155, 150, 135]
])

# Example 2: Create a 3D array (multi-band raster: 5x5 pixels, 3 bands)
# Simulate Red, Green, Blue (RGB) bands for a satellite image
rgb_array = np.array([
    # Red band (band 0)
    [[210, 205, 200, 195, 190],
     [205, 200, 195, 190, 185],
     [200, 195, 190, 185, 180],
     [195, 190, 185, 180, 175],
     [190, 185, 180, 175, 170]],
    # Green band (band 1)
    [[190, 185, 180, 175, 170],
     [185, 180, 175, 170, 165],
     [180, 175, 170, 165, 160],
     [175, 170, 165, 160, 155],
     [170, 165, 160, 155, 150]],
    # Blue band (band 2)
```

```

[[170, 165, 160, 155, 150],
 [165, 160, 155, 150, 145],
 [160, 155, 150, 145, 140],
 [155, 150, 145, 140, 135],
 [150, 145, 140, 135, 130]]

])

# Example 3: Create arrays with placeholder values (useful for preallocating space)
# Create a 10x10 array of zeros (simulate empty raster)
empty_raster = np.zeros((10, 10), dtype=np.float32) # Use float32 for geospatial data
# Create a 8x8 array of ones (simulate uniform reflectance)
uniform_reflectance = np.ones((8, 8), dtype=np.float32) * 0.5 # 0.5 = 50% reflectance
print("\n--- NumPy Array Creation (Geospatial Examples) ---")
print(f'DEM Array (2D, single-band):\n{dem_array}')
print(f'\nRGB Array (3D, multi-band) Shape: {rgb_array.shape}') # Shape: (bands, height,
width)
print(f'Empty Raster (zeros) Shape: {empty_raster.shape}')
print(f'Uniform Reflectance Array:\n{uniform_reflectance}')

```

2.4.2 NumPy Array Attributes

Array attributes provide key information about raster data (e.g., dimensions, data type) that is essential for geospatial workflows.

```

# Get attributes of the DEM array (single-band raster)
print("\n--- DEM Array Attributes ---")
print(f'Shape (rows, columns): {dem_array.shape}') # Geospatial: (height,
width)
print(f'Number of dimensions: {dem_array.ndim}') # 2 = single-band, 3 =
multi-band
print(f'Data type: {dem_array.dtype}') # Ensures compatibility (e.g.,
float32 for reflectance)
print(f'Total number of elements: {dem_array.size}') # Total pixels in the
raster
print(f'Minimum value: {dem_array.min()}') # Minimum elevation (DEM
example)
print(f'Maximum value: {dem_array.max()}') # Maximum elevation
(DEM example)
print(f'Mean value: {dem_array.mean():.2f}') # Average elevation (DEM

```

example)

```
# Get attributes of the RGB array (multi-band raster)
print("\n--- RGB Array Attributes ---")
print(f'Shape (bands, height, width): {rgb_array.shape}') # Standard for
multi-band
print(f'Data type: {rgb_array.dtype}') # Integer for RGB (0–255)
print(f'Mean of Red band (band 0): {rgb_array[0].mean():.2f}') # Mean of
first band
print(f'Mean of Green band (band 1): {rgb_array[1].mean():.2f}') # Mean
of second band
```

2.4.3 Array Indexing & Slicing (Extract Regions of Interest)

Indexing (select single elements) and slicing (select subarrays) are used to extract regions of interest (ROIs) from raster data (e.g., a field within a satellite image, a city within a DEM).

```
# Example 1: Indexing (select single pixels from the DEM)
# Geospatial note: Raster indices are (row, column) → corresponds to (y, x)
in spatial coordinates
top_left_pixel = dem_array[0, 0] # Top-left corner (row 0, column 0)
center_pixel = dem_array[2, 2] # Center (row 2, column 2)
bottom_right_pixel = dem_array[-1, -1] # Bottom-right corner (last row,
last column)
print("\n--- DEM Array Indexing (Single Pixels) ---")
print(f'Top-left pixel (elevation): {top_left_pixel}m')
print(f'Center pixel (elevation): {center_pixel}m')
print(f'Bottom-right pixel (elevation): {bottom_right_pixel}m')
# Example 2: Slicing (extract a region of interest: ROI)
# Extract a 3x3 subarray from the DEM (rows 1–3, columns 1–3)
roi_dem = dem_array[1:4, 1:4] # Slicing: [start_row:end_row,
start_col:end_col]
print("\n--- DEM Region of Interest (ROI: 3x3) ---")
print(roi_dem)
# Example 3: Slice a specific band from a multi-band array
# Extract the Green band (band 1) from the RGB array
green_band = rgb_array[1, :, :] # [band_index, all_rows, all_columns]
# Extract a 3x3 ROI from the Red band (band 0)
roi_red = rgb_array[0, 1:4, 1:4]
print("\n--- Multi-Band Slicing ---")
print(f'Green Band (shape): {green_band.shape}')
print(f'Red Band ROI (3x3):\n{roi_red}')
```

3 Basic data processing

The python libraries we need to use include numpy (basic module for python scientific computation), matplotlib (data visualization), and PIL.Image (read, write and transform images).

3.1 Process with remote sensing image data

In this part, we will give instructions including remote sensing reading, writing, transformation and visualization.

The remote sensing data is in folder of “rs_data”, which contains 10 high resolution remote sensing images.

- 1) Create a new python file “process_rs”
- 2) Import the required python libraries.

```
from PIL import Image  
import numpy as np
```

- 3) Load image

```
img=Image.open(r"Your data path/airplane46.tif")
```

- 4) print image information

```
print('size:',img.size)  
print('mode:',img.mode)  
size: (256, 256)  
mode: RGB
```

- 5) Show image

```
img.show('airplane')
```



6) Transform the image

The library PIL.Image provides many functions for image transformation. For example, you can use `img=img.resize((100,100))` to resize the image. Here I list some functions. You can try them and use `img.show()` to display the transformed image.

Some available transform functions:

```
img=img.rotate(90)
img=img.transpose(Image.FLIP_LEFT_RIGHT)
img=img.transpose(Image.FLIP_TOP_BOTTOM)
img=img.crop((0,0,50,50))
```

7) calculate statistics of images

convert the image into numpy format and use numpy to calculate statistics.

Convert the image:

```
img_array=np.array(img)
```

calculate the minimum, maximum, mean and standard deviation, respectively.

```
print('min:',np.min(img_array))
print('max:',np.max(img_array))
print('mean:',np.mean(img_array))
print('std:',np.std(img_array))
```

3.2 Process with csv file and data

A CSV (Comma Separated Values) file is a type of plain text file that uses specific structuring to arrange tabular data. It's a simple format for storing data in a tabular structure. Sometimes, we save the geographical data in form of matrix with csv format. Therefore, here we will give a brief introduction on how to process with csv file.

In the folder “csv_data”, we provide some csv files. In each csv file, it provides a matrix where each row indicates the acquisition date and each column indicates the spectral band associated, namely blue (B2), green (B3), red (B4), the four vegetation red edge (B5, B6, B7 and B8A) and the two short wave infrared (SWIR) (B11 and B12). The last column contains information regarding the condition of the pixel: clear (0), cloud (1), shadow (2) or snow (3).

Here are instructions about reading csv file and visualization.

- 1) import libraries

```
import numpy as np
import matplotlib.pyplot as plt
```

- 2) load data

```
data = np.loadtxt(r'Your data path/0.csv', delimiter=',', skiprows=1)
```

- 3) print some info

```
print(data.shape)
print(data[0:5,:])
print('mean:',np.mean(data))
```

- 4) visualize the blue band data with time series

```
plt.plot(data[:,2])
plt.show()
```

4 More practice

4.1 Process for multispectral images

The above introduction is about image with RGB bands. For geospatial data, it usually contains more bands. To process it, we need to use python library `tifffile`. (Please follow section 1.1.3 to install this library).

In the folder “ms_data”, we provide some multi-spectral data with resolution 10 meters and 30 meters. The following is an example to process it.

- 1) Create new file “process_ms.py”
- 2) Import `tifffile` library and read `img`

```
import tifffile as tiff  
img=tiff.imread(r"D:/codes/lab1_data/rs_data/rs_10m_1.tif")
```

- 3) As the `Image` library just supports image with RGB bands, just select three bands to convert and display. Then you can process it in the way we introduce in section 2.1.

```
img=Image.fromarray(img[:, :, [2,1,0]])  
img.show('ms data')
```

4.2 More

- 1) Try to calculate the minimum, maximum and other statistics on all images.
- 2) Try to save the transformed image to the disk.
- 3) Read all the csv files and concatenate all the data into a larger matrix and save it